SCH- MGMT 661:
Applications of Artificial Intelligence Models

Case: Transfer Learning with MobileNet2

Isenberg School of Management

Instructor: Indika Dissanayake

Case: Transfer Learning with MobileNet2 (CIFAR-10)

- Last module we trained a CNN from scratch on CIFAR-10. This module, we'll reuse a pretrained ImageNet model (MobileNetV2/V3) and adapt it to the same dataset.
- What we'll do
  - Load CIFAR-10 and resize images to 96×96
  - Build a model with a frozen pretrained base + new classification head
  - Train only the new head (feature extraction baseline)
  - Evaluate (accuracy, loss, confusion matrix)
  - Fine-tune: unfreeze a few top layers and train with a small learning rate
  - Try data augmentation to improve generalization (flip/rotate), then compare results

Model Building

- Load the base model- MobileNet2 pretrained on ImageNet
  - Input shape: (96, 96, 3)
  - Exclude the original top classification layer (include_top=False)
  - Freeze weights

```
base_model = MobileNetV2(
    input_shape=(96, 96, 3),
    include_top=False,
    weights='imagenet’
)
base_model.trainable = False
```

- Add the base model
- Add a new “head”
  - Global Average Pooling layer (to reduce feature maps)
  - Dense(128, activation='relu’)
  - Dropout(0.3) to reduce overfitting
  - Output layer with 10 softmax nodes for CIFAR-10 classes

```
inputs = layers.Input(shape=(96, 96, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(10,activation='softmax')(x)

model = models.Model(inputs, outputs)
```

Compile and Train the model

Compile the Model:

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

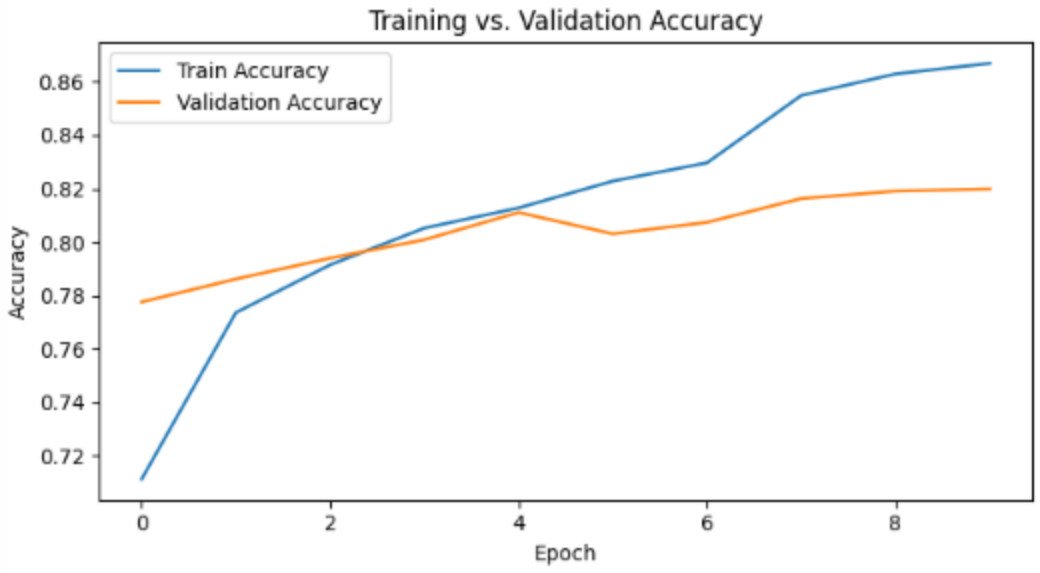
Train the Model:

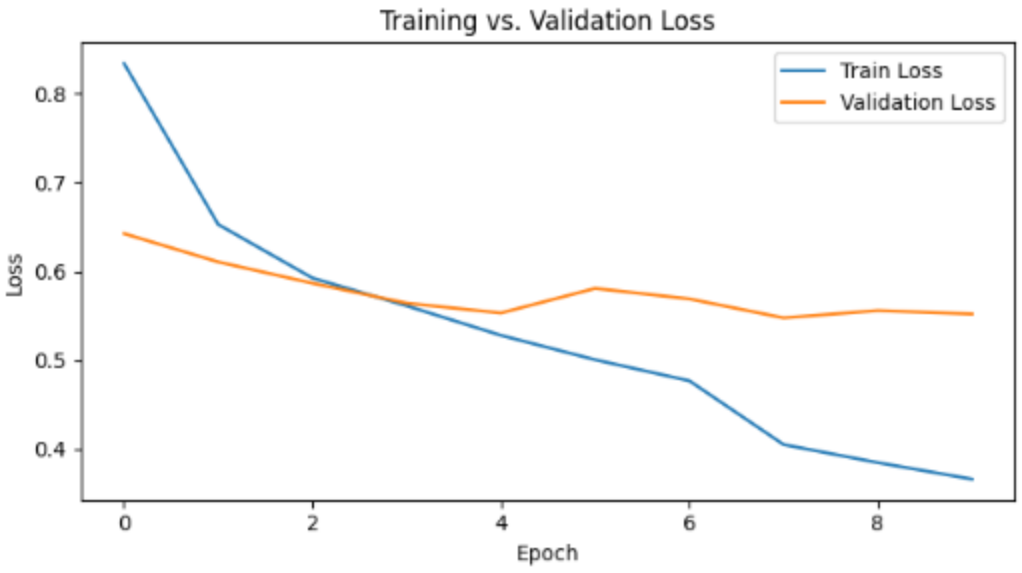
```
history = model.fit(
    x_train_resized,
    y_train,
    batch_size=32,
    epochs=10,
    shuffle=True,
    validation_split=0.2,
    callbacks=callbacks
)
```

Evaluate the Model

```
# Evaluate on test data
test_loss, test_accuracy = model.evaluate(x_test_resized, y_test, verbose=0)
```

```
Final Test Loss: 0.5633
Final Test Accuracy: 0.8103
```





Fine-Tuning the Model

- When? After accuracy plateaus with the frozen base model.
- Why? To unlock more learning by allowing some deeper pretrained layers to adjust to the new dataset.
- How?
  - Unfreeze base model (base_model.trainable = True)
  - Keep most layers frozen (e.g., only unfreeze last 10-20)
  - Recompile with a low learning rate
  - Train for a few more epochs (start small: epochs=5)
  - Use only if your system can handle it (longer training, more memory).

```
# Unfreeze the base model
base_model.trainable = True

# Freeze all layers except the last 20
for layer in base_model.layers[:-20]:
    layer.trainable = False


# Recompile with a low learning rate
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

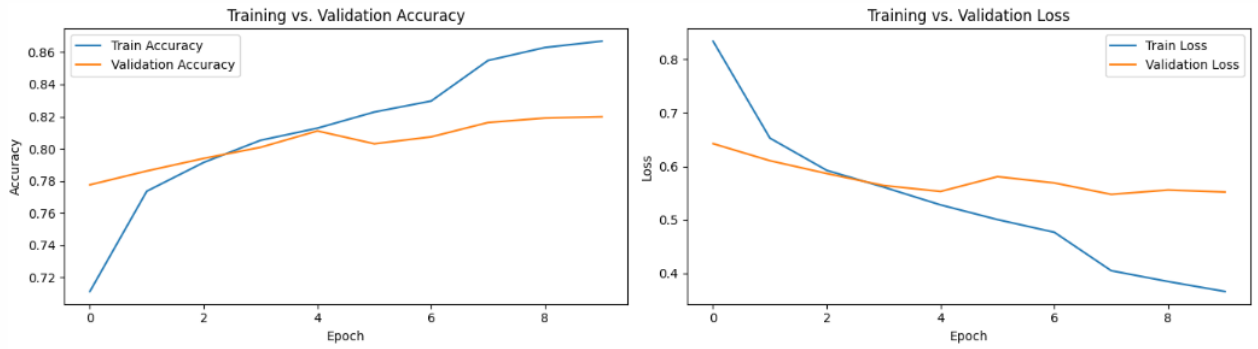
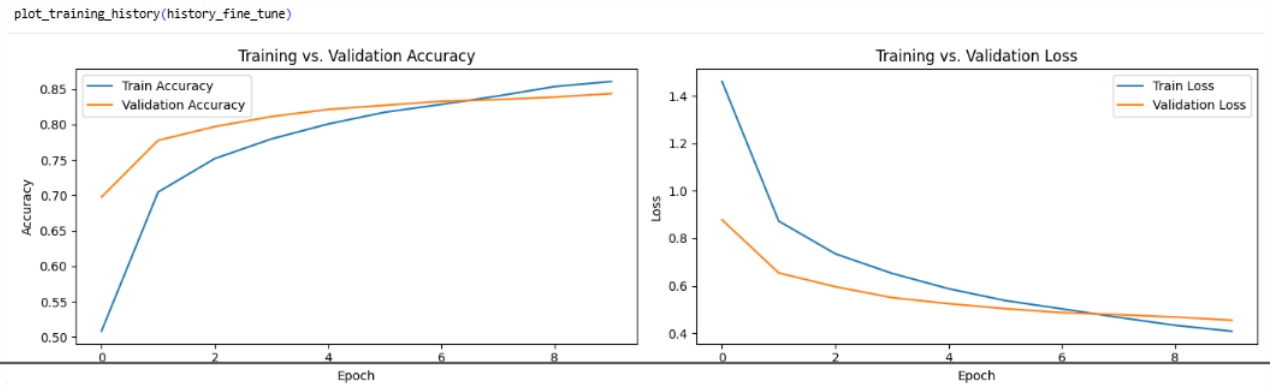
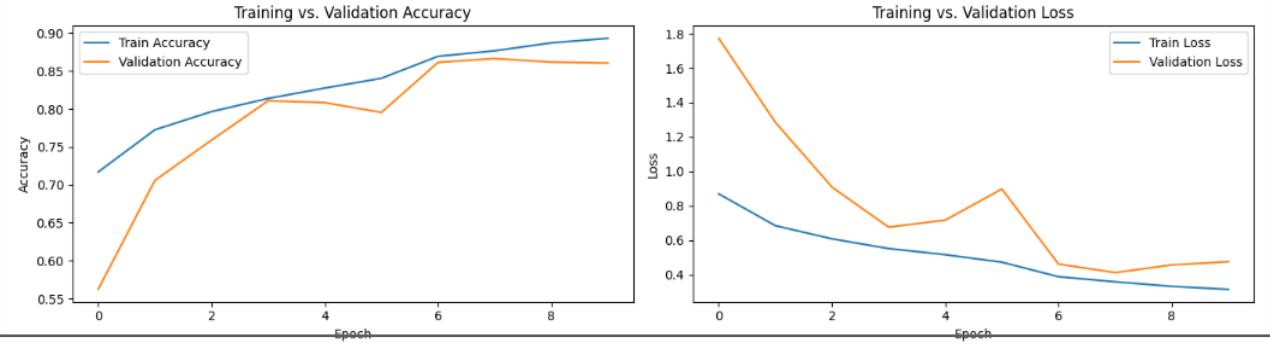
Base Model with Data Augmentation

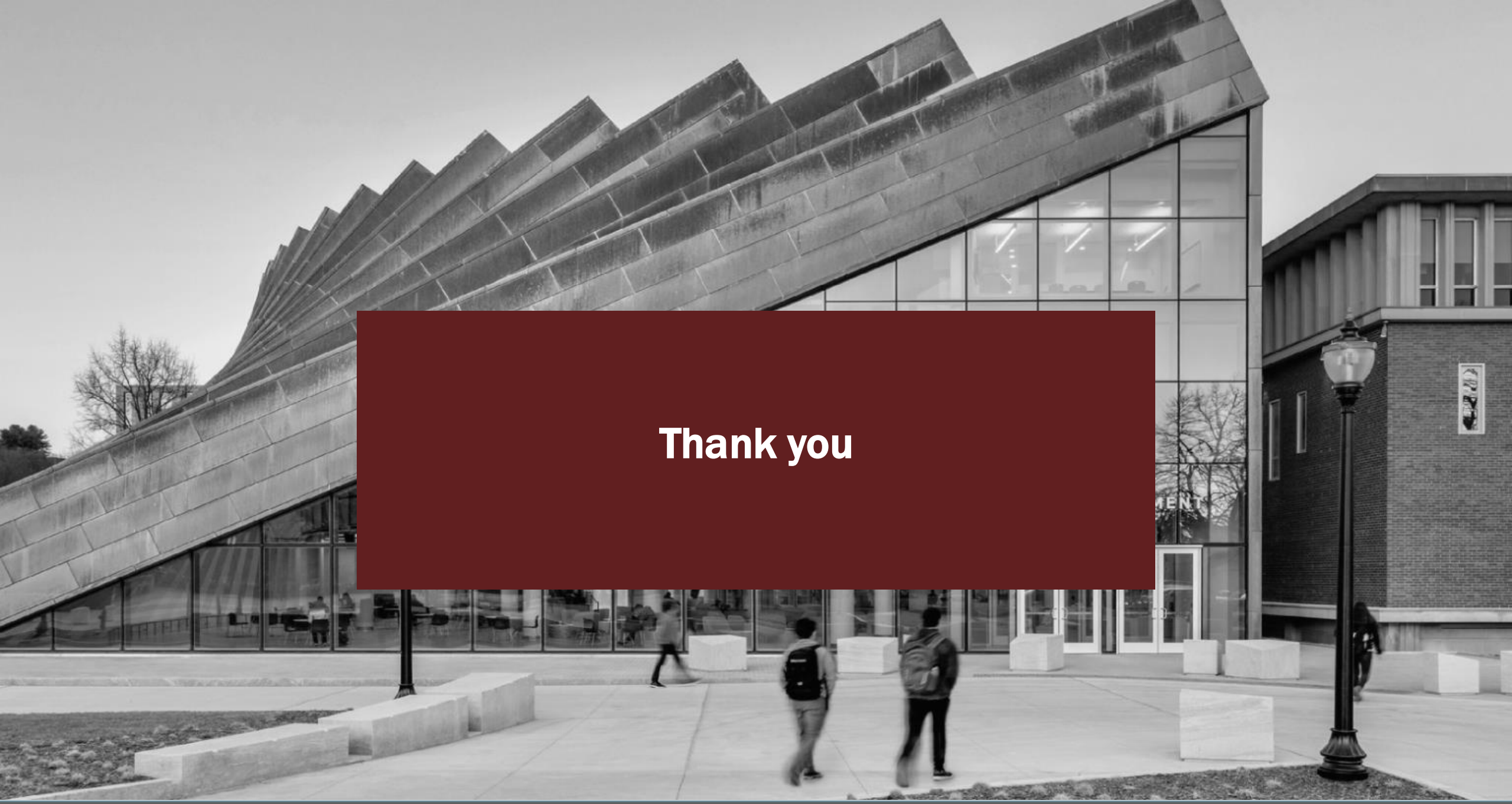
- How?
  - Apply augmentation inside the model
  - Keep the pretrained base frozen (feature extraction phase).
  - Pass augmented images → base model → pooling → dense head.
- Why ?
  - Increases data diversity without collecting more data.
  - Reduces overfitting.
  - Runs on-the-fly (no extra storage for data)

```
inputs = layers.Input(shape=(96, 96, 3))
x = data_augmentation(inputs) # augmentation
x = base_model(x, training=False)# Base frozen
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs =
    layers.Dense(10,activation='softmax')(x)
```

```
model = models.Model(inputs, outputs)
```

Model Comparison

Base model: Validation accuracy plateaus early → limited learning with frozen layers.	
Fine-tuned model: Improved accuracy and lower loss → better adaptation to CIFAR-10.	
Base with augmentation: More stable validation curve → reduced overfitting, stronger generalization.	



Thank you

Reference

- https://www.tensorflow.org/tutorials/images/transfer_learning
- https://keras.io/api/applications/